- HMS Dashboard — Complete Project Documentation
  - Table of Contents
  - 1. Project Overview
    - Key Capabilities
  - 2. Technology Stack
    - Frontend
    - Backend
  - 3. Project Structure
  - 4. Architecture & Design
    - Server Architecture
    - Client Architecture
  - 5. Backend Modules (Server)
  - 6. Frontend Modules (Client)
    - 6.1 Public Pages
    - 6.2 Admin Pages (Admin only)
    - 6.3 Hospital / Reception Pages (Receptionist, Admin)
    - 6.4 Doctor Pages (Doctor, Admin)
    - 6.5 Pharmacy Pages (Pharmacist, PHARMA_MASTER, Admin)
    - 6.6 Lab Pages (Lab Technician, LAB_MASTER, Admin)
    - 6.7 Payroll & HR Pages (Admin, HR)
  - 7. Authentication & RBAC
    - Authentication Flow
    - Defined Roles
    - Middleware Chain
  - 8. API Reference
    - Auth (/api/auth)
    - Patients (/api/patients)
    - Appointments (/api/appointments)
    - IPD (/api/ipd)
    - OPD (/api/opd)
    - Billing (/api/billing)
    - Pharmacy (/api/pharmacy)
    - Pharmacy Billing (/api/pharmacy-billing)
    - Lab (/api/lab)
    - Payroll (/api/payroll)
    - Admin (/api/admin) — All routes require Admin role
    - Other Routes

- 9. Database
  - Key Tables (Inferred from modules)
- 10. Middleware Pipeline
  - Caching Middleware
- 11. Reusable Components
  - Common UI Components (client/src/components/common/)
  - Print Templates (client/src/components/print/)
- 12. DevOps & Deployment
  - Docker
  - Kubernetes (k8s/)
  - Nginx Configuration
- 13. Testing
- 14. Environment Configuration
  - Server .env Variables
  - NPM Scripts
- 15. Role Workflows
  - Admin Day-in-Life Summary
- Appendix: Existing Documentation Files

HMS Dashboard — Complete Project Documentation

Version: 2.0.0
Last Updated: 2026-02-23
Architecture: Monorepo (Client + Server)

Table of Contents

1. Project Overview
2. Technology Stack
3. Project Structure
4. Architecture & Design
5. Backend Modules (Server)
6. Frontend Modules (Client)
7. Authentication & RBAC

8. API Reference
9. Database
10. Middleware Pipeline
11. Reusable Components
12. DevOps & Deployment
13. Testing
14. Environment Configuration
15. Role Workflows

---

1. Project Overview

HMS Dashboard is a production-grade **Hospital Management System** designed for multi-specialty hospitals. It covers patient lifecycle management, clinical workflows, pharmacy operations, laboratory management, billing, payroll/HR, and administrative oversight.

Key Capabilities

- **Patient Management** — Registration, OPD/IPD, Smart Cards, Medical Records
- **Doctor Module** — Consultations, Medical Notes, Lab Requests, Prescriptions
- **Pharmacy** — Stock Management, Vendor/Supplier Management, Billing, Purchase Orders
- **Laboratory** — Test Master, Test Entry, Lab Billing, Request Approval Workflow
- **Billing** — Unified Billing, Pharmacy Billing, Appointment Transactions
- **Payroll & HR** — Employee Master, Attendance, Salary Processing, Payslip Generation
- **Admin Panel** — Dashboard Analytics, User Management, Role/Permission Control, Audit Logs, System Monitoring
- **Reception** — Patient Registration, Appointments, Doctor Status/Directory
- **Reports** — Daily OP, Patient History, Stock Reports, Purchase Reports, Daily Sales

---

2. Technology Stack

Frontend

Layer	Technology
Framework	React 19 (Vite)
Language	JavaScript (JSX)
Styling	Tailwind CSS v4
Routing	React Router DOM v7
Charts	Recharts
Icons	Lucide React
Animations	Framer Motion
Data Grid	AG Grid React
HTTP Client	Axios
Notifications	React Hot Toast
PDF Generation	html2pdf.js, react-to-print
QR Codes	qrcode.react

Backend

Layer	Technology
Runtime	Node.js
Framework	Express.js v5
Database	MySQL 8.0 (mysql2 driver)
Auth	JWT (jsonwebtoken) + bcryptjs
Security	Helmet, express-rate-limit, CORS
Logging	Winston + Morgan
Compression	gzip (compression)
PDF Reports	Puppeteer
Config	dotenv

3. Project Structure

```
hms-dashboard/
├── client/                        # React Frontend (Vite)
│   ├── src/
│   │   ├── App.jsx                # Root component + all route definitions
│   │   ├── main.jsx               # React entry point
│   │   ├── index.css              # Global Tailwind styles
│   │   ├── context/               # React Context Providers
│   │   │   ├── AuthContext.jsx    # Auth state (user, token, login/logout)
│   │   │   ├── PatientContext.jsx # Shared patient state
│   │   │   └── ToastContext.jsx   # Global toast notifications
│   │   ├── layouts/
│   │   │   └── DashboardLayout.jsx # Sidebar + Header wrapper
│   │   ├── components/
│   │   │   ├── common/            # Reusable UI (Button, Card, Badge, Input,
DataTable)
│   │   │   │   ├── MasterSearch.jsx     # Global search bar
│   │   │   │   ├── MedicineSearch.jsx   # Medicine autocomplete
│   │   │   │   ├── PatientSearch.jsx    # Patient lookup
│   │   │   │   ├── PrintLayout.jsx      # Print wrapper
│   │   │   │   └── DashboardStack.jsx   # Dashboard widget stack
│   │   │   ├── layout/
│   │   │   │   ├── Sidebar.jsx     # Role-aware navigation sidebar
│   │   │   │   ├── Header.jsx      # Top bar with search + user menu
│   │   │   │   └── PageTransition.jsx
│   │   │   └── print/
│   │   │       ├── InvoicePrint.jsx
│   │   │       ├── PatientHistoryPrint.jsx
│   │   │       └── PharmacyBillPrint.jsx
│   │   ├── pages/
│   │   │   ├── public/            # Landing, Login, RoleSelection
│   │   │   ├── admin/             # Monitoring, ActivityLogs, Dashboard
│   │   │   ├── dashboard/admin/   # User CRUD, Role Creation
│   │   │   ├── hospital/          # Reception, Appointments, IPD, OPD, Billing,
etc.
│   │   │   ├── doctor/            # PatientList, MedicalNotes, LabRequest,
Prescriptions
│   │   │   ├── pharmacy/          # Dashboard, Stock, Vendors, Billing, Reports,
Orders
│   │   │   ├── lab/               # Dashboard, ProductMaster, Indents, TestEntry
│   │   │   ├── lab-master/        # TestMaster, DoctorEntry
│   │   │   ├── lab-entry/         # TestEntry, LabBilling, TestReport
│   │   │   ├── payroll/           # Dashboard, EmployeeMaster, Attendance, Salary
│   │   │   ├── reception/         # DoctorStatus, DoctorProfile
│   │   │   └── billing/           # UnifiedBilling
│   │   ├── services/
│   │   └── lib/
│   ├── public/
│   ├── dist/                      # Production build output
│   ├── Dockerfile
│   ├── nginx.conf
│   └── package.json
│
```

```
├── server/                          # Express Backend
│   ├── server.js                    # Entry point (Cluster mode support)
│   ├── src/
│   │   ├── app.js                   # Express app config, middleware, route
mounting
│   │   ├── config/
│   │   │   ├── db.js                # MySQL connection pool (100 connections)
│   │   │   ├── jwt.js               # JWT secret config
│   │   │   └── role_modules.js      # Role → Module access mapping
│   │   ├── middlewares/
│   │   │   ├── auth.middleware.js   # verifyToken, authorizeRoles,
checkPermission, authorizeModule
│   │   │   ├── cache.middleware.js  # In-memory response caching + invalidation
│   │   │   ├── error.middleware.js  # Global error handler
│   │   │   └── validate.middleware.js # Input sanitization (XSS prevention)
│   │   ├── utils/
│   │   │   ├── logger.js            # Winston logger (file + console)
│   │   │   ├── auditLogger.js       # DB-level audit trail logging
│   │   │   ├── AppError.js          # Custom error class
│   │   │   └── catchAsync.js        # Async error wrapper
│   │   ├── modules/                 # 23 feature modules (see Section 5)
│   │   ├── templates/               # PDF report templates
│   │   └── scripts/                 # DB migration & utility scripts
│   ├── routes/                      # Legacy route files
│   ├── tests/
│   │   ├── load/                    # Load testing (autocannon)
│   │   ├── sanity/                  # Sanity checks
│   │   └── e2e/                     # End-to-end verification
│   ├── hms.sql                      # Full database schema dump
│   ├── Dockerfile
│   └── package.json
│
├── k8s/                             # Kubernetes manifests
├── documentation/                   # API Guide, Project History, Postman
Collection
├── docker-compose.yml               # 3-service stack (backend, frontend, db)
├── Admin Workflow.txt
├── Doctor Workflow.txt
├── Receptionist Workflow.txt
├── Lab Technician Workflow.txt
├── Pharma Workflow.txt
└── PAYROLL_MODULE_BLUEPRINT.md
```

---

4. Architecture & Design

Server Architecture

- **Cluster Mode** — Optional multi-worker process mode via Node.js `cluster` module. Controlled by `ENABLE_CLUSTER=true` in `.env`. Auto-restarts crashed

workers.

- **Graceful Shutdown** — Handles `SIGTERM`/`SIGINT` with connection draining. Force-kills after 30s timeout.
- **Connection Pool** — MySQL pool with 100 max connections, keep-alive enabled, IST timezone.
- **Modular Backend** — Each feature is a self-contained module with its own `routes`, `controller`, and `service` files.

Client Architecture

- **Context-Based State** — `AuthContext`, `PatientContext`, `ToastContext` wrap the app.
- **Role-Aware Routing** — `ProtectedRoute` component checks `user.role` against `allowedRoles` array. Admin bypasses all checks.
- **Layout System** — `DashboardLayout` wraps authenticated pages with Sidebar + Header. Public pages (Login, Landing) render outside the layout.

---

5. Backend Modules (Server)

All modules live in `server/src/modules/`. Each module has: `*.routes.js`, `*.controller.js`, and optionally `*.service.js`.

#	Module	Base Route	Description
1	**auth**	`/api/auth`	Login, get roles, get current user (`/me`)
2	**patients**	`/api/patients`	CRUD patients, update status
3	**medical-records**	`/api/medical-records`	Create clinical records, get patient history
4	**prescriptions**	`/api/prescriptions`	Save/get prescriptions, update status
5	**tests**	`/api/tests`	Lab test master CRUD
6	**inventory**	`/api/inventory`	Products, indents, issues, lab patients

#	Module	Base Route	Description
7	**admin**	`/api/admin`	Dashboard stats, analytics, roles, permissions, users, audit logs, search
8	**appointments**	`/api/appointments`	Appointment CRUD, transactions
9	**ipd**	`/api/ipd`	Bed management, admissions, discharge, encounters
10	**opd**	`/api/opd`	OPD visits: register, complete, delete
11	**billing**	`/api/billing`	Invoices: create, list, payment recording, delete
12	**lab**	`/api/lab`	Doctor referrals, test entry, lab billing, requests (approval workflow), reports
13	**timeline**	`/api/timeline`	Patient timeline events
14	**pharmacy**	`/api/pharmacy`	Products, vendors, purchases, enquiries, request approval, stock alerts, suppliers, orders
15	**pharmacy-billing**	`/api/pharmacy-billing`	Pharmacy bill creation, returns, bill/credit reports, patient financials
16	**daily-op**	`/api/daily-op`	Daily outpatient report
17	**doctor**	`/api/doctors`	Doctor list, details, status update
18	**search**	`/api/search`	Global RBAC-filtered search across entities
19	**payroll**	`/api/payroll`	Employees, attendance, shifts, salaries, payslip generation
20	**logs**	`/api/logs`	System activity logs
21	**reports**	`/api/reports`	Generic report generation (PDF via Puppeteer)
22	**user-management**	`/api/user-management`	Admin-only CRUD for users, roles, modules, staff

#	Module	Base Route	Description
23	**document**	—	Document handling

6. Frontend Modules (Client)

6.1 Public Pages

Page	Path	Description
Landing	`/`	Hospital landing page
Login	`/login`	Email/password auth with role selection

6.2 Admin Pages (Admin only)

Page	Path	Description
Admin Dashboard	`/dashboard`	KPI widgets, charts, real-time stats
Monitoring	`/admin/monitoring`	Live audit feed, active users, role filter
Activity Logs	`/admin/logs`	Searchable audit trail
User List	`/admin/users/list`	User management table
Create User	`/admin/users/create`	New user onboarding
Edit User	`/admin/users/edit/:id`	Update user details
Create Role	`/admin/roles/create`	Define new roles

6.3 Hospital / Reception Pages (Receptionist, Admin)

Page	Path	Description
Reception	`/hospital/reception`	Patient registration & queue
Appointments	`/hospital/appointments`	Booking & management
Smart Cards	`/hospital/smart-cards`	Patient smart card generation
Billing	`/hospital/billing`	Hospital billing interface
Unified Billing	`/hospital/unified-billing`	Consolidated billing view
Appointment Transactions	`/hospital/appointment-transactions`	Financial transactions
Doctor Directory	`/hospital/doctor-directory`	Browse doctors
Doctor Status	`/reception/doctors`	Real-time doctor availability
Doctor Profile	`/reception/doctors/:id`	Individual doctor profile
Daily OP Report	`/hospital/reports/daily-op`	Daily outpatient report

6.4 Doctor Pages (Doctor, Admin)

Page	Path	Description
Consultations	`/doctor/consultations`	Assigned patient list
Medical Notes	`/doctor/notes/:visitId`	Clinical notes entry
Lab Request	`/doctor/lab-request`	Order lab tests
Lab Results	`/doctor/lab-results`	View test results
Prescriptions	`/doctor/prescriptions`	Prescription management

Page	Path	Description
Prescription Entry	`/hospital/prescription`	Write prescriptions
IPD	`/hospital/ipd`	In-patient management
OPD	`/hospital/opd`	Outpatient visits
Medical Records	`/hospital/medical-records`	Patient history viewer
Encounters	`/hospital/encounters`	Clinical encounters
Patient History	`/hospital/reports/patient-history`	Full patient timeline

6.5 Pharmacy Pages (Pharmacist, PHARMA_MASTER, Admin)

Page	Path	Description
Dashboard	`/pharmacy/dashboard`	Pharmacy KPIs & alerts
Patient Details	`/pharmacy/patient-details`	Patient pharmacy records
Vendor Details	`/pharmacy/vendor-details`	Vendor management
Stock Entry	`/pharmacy/stock-entry`	Add/manage stock
Purchase Entry	`/pharmacy/purchase-entry`	Record purchases
Enquiry Entry	`/pharmacy/enquiry-entry`	Price enquiries
Pending Requests	`/pharmacy/requests/pending`	Approve/reject requests
Billing	`/pharmacy/billing`	Pharmacy billing
Return Billing	`/pharmacy/billing/return`	Process returns
Stock Report	`/pharmacy/reports/stock`	Stock analysis
Purchase Report	`/pharmacy/reports/purchase`	Purchase analysis
Daily Report	`/pharmacy/reports/daily`	Daily sales report
Suppliers	`/pharmacy/suppliers`	Supplier management
Stock Orders	`/pharmacy/stock-orders`	Purchase orders list

Page	Path	Description
Create Order	`/pharmacy/orders/create`	New purchase order
Order Details	`/pharmacy/orders/:id`	Order tracking

6.6 Lab Pages (Lab Technician, LAB_MASTER, Admin)

Page	Path	Description
Dashboard	`/lab/dashboard`	Lab workload overview
Patients	`/lab/patients`	Lab patient records
Product Master	`/lab/products`	Lab consumables
Indent Entry	`/lab/indents`	Requisition consumables
Product Issue	`/lab/issues`	Issue consumables
Pending Requests	`/lab/requests/pending`	Approve lab requests (LAB_MASTER)
Test Master	`/lab-master/test-master`	Define tests & parameters
Doctor Entry	`/lab-master/doctor-entry`	Manage referring doctors
Test Entry	`/lab-entry/test-entry`	Enter test results
Lab Billing	`/lab-entry/billing`	Lab test billing
Test Report	`/lab-entry/report`	View/print test reports

6.7 Payroll & HR Pages (Admin, HR)

Page	Path	Description
Dashboard	`/payroll/dashboard`	Payroll overview stats
Employee Master	`/payroll/employees`	Employee CRUD
Attendance	`/payroll/attendance`	Daily attendance marking
Salary Details	`/payroll/salaries`	View/edit salary structure

Page	Path	Description
Salary Processing	`/payroll/process`	Monthly payroll run + payslip generation

7. Authentication & RBAC

Authentication Flow

1. User submits email + password to `POST /api/auth/login`
2. Server validates credentials (bcrypt hash comparison)
3. JWT issued with payload: `{ id, username, role, iat, exp }`
4. Client stores token and attaches as `Authorization: Bearer <token>`
5. `GET /api/auth/me` validates token and returns current user

Defined Roles

Role	Module Access
Admin	All modules (dashboard, users, logs, payroll, pharmacy, lab, reception, ipd, opd, billing, reports, appointments)
Doctor	dashboard, opd, lab, ipd, reports, appointments
Receptionist	dashboard, reception, opd, appointments, billing
Pharmacist	dashboard, pharmacy
PHARMA_MASTER	dashboard, pharmacy
Lab Technician	dashboard, lab
LAB_MASTER	dashboard, lab
Accountant	dashboard, billing
Nurse	dashboard, ipd
HR	dashboard, payroll
Trainee	dashboard

Middleware Chain

```
verifyToken → authorizeRoles(...roles) → controller
```

- `verifyToken` — Decodes JWT, attaches `req.user`
- `authorizeRoles` — Checks if `req.user.role` is in allowed list
- `checkPermission(module, action)` — DB-level granular permission check
- `authorizeModule(moduleKey)` — Checks `module_access` array from JWT

---

8. API Reference

Auth (`/api/auth`)

Method	Endpoint	Auth	Description
POST	`/login`	No	Authenticate user, return JWT
GET	`/roles`	No	Get roles for login dropdown
GET	`/me`	Yes	Get current user profile

Patients (`/api/patients`)

Method	Endpoint	Description
GET	`/`	List all patients
GET	`/:id`	Get patient by ID
POST	`/`	Register new patient
PATCH	`/:id/status`	Update patient status

Appointments (`/api/appointments`)

Method	Endpoint	Roles	Description
GET	`/`	Any auth	List appointments
POST	`/`	Doctor, Receptionist, Nurse	Create appointment
PUT	`/:id/status`	Doctor, Receptionist, Nurse	Update status
DELETE	`/:id`	Doctor, Receptionist, Nurse	Cancel appointment
GET	`/transactions`	Any auth	List transactions
POST	`/transactions`	Any auth	Record transaction

IPD (`/api/ipd`)

Method	Endpoint	Roles	Description
GET	`/beds`	Any auth	Get bed status
GET	`/admissions`	Any auth	List admissions
POST	`/admissions`	Doctor, Nurse, Receptionist	Admit patient
POST	`/admissions/:id/discharge`	Doctor, Nurse, Receptionist	Discharge patient
GET	`/encounters`	Any auth	Encounters dashboard

OPD (`/api/opd`)

Method	Endpoint	Description
GET	`/`	List OPD visits
POST	`/`	Register visit
PUT	`/:id/complete`	Complete visit
DELETE	`/:id`	Delete visit

Billing (`/api/billing`)

Method	Endpoint	Description
GET	`/`	List invoices
GET	`/:id`	Get invoice detail
POST	`/`	Create invoice
POST	`/:id/payment`	Record payment
DELETE	`/:id`	Delete invoice

Pharmacy (`/api/pharmacy`)

Method	Endpoint	Description
GET	`/stats`	Dashboard statistics
GET/POST/PUT/DELETE	`/vendors`	Vendor CRUD
GET/POST/PUT/DELETE	`/products`	Product (stock) CRUD
GET/POST	`/purchases`	Purchase records
GET/POST	`/enquiries`	Price enquiries
POST	`/requests`	Doctor creates med request
GET	`/requests/pending`	Pending approval queue
PUT	`/requests/:id/status`	Approve/reject request
GET	`/alerts`	Stock alerts
PUT	`/alerts/:id/acknowledge`	Acknowledge alert
GET/POST	`/suppliers`	Supplier management
GET/POST	`/orders`	Purchase orders
GET	`/orders/:id`	Order details
PUT	`/orders/:id/submit`	Submit order
POST	`/orders/:id/receive`	Receive stock
GET	`/reports/daily`	Daily sales report

Pharmacy Billing (`/api/pharmacy-billing`)

Method	Endpoint	Description
POST	`/bill`	Create pharmacy bill
POST	`/return`	Process return
GET	`/reports/bill`	Bill report
GET	`/reports/credit`	Credit report
GET	`/patient-financials/:patientId`	Patient financial profile

Lab (`/api/lab`)

Method	Endpoint	Description
GET/POST/PUT/DELETE	`/doctors`	Referring doctor CRUD
POST	`/test-entry`	Create test entry
GET	`/test-entry`	List entries
GET	`/test-entry/:id/print`	Print lab report
GET	`/test-entry/:id/download`	Download PDF
POST	`/billing`	Create lab bill
GET	`/billing`	List lab bills
POST	`/request`	Create lab request (Doctor)
GET	`/requests/pending`	Pending approval (LAB_MASTER)
PUT	`/request/:id/status`	Approve/reject
GET	`/patients`	Search lab patients
GET	`/tests`	List available tests
GET	`/next-id`	Next test ID

Payroll (`/api/payroll`)

Method	Endpoint	Roles	Description
GET	`/employees`	Any auth	List employees
GET	`/employees/:id`	Any auth	Employee details
POST	`/employees`	Admin, HR	Create employee
PUT	`/employees/:id`	Admin, HR	Update employee
GET	`/shifts`	Any auth	Available shifts
GET	`/attendance/daily`	Any auth	Daily attendance
POST	`/attendance/mark`	Admin, HR	Mark attendance
GET	`/salaries`	Admin, HR	List salaries
GET	`/salaries/:id`	Admin, HR	Employee salary
PUT	`/salaries/:id`	Admin, HR	Update salary
GET	`/payslips`	Admin, HR	List payslips
POST	`/generate`	Admin, HR	Generate payroll

Admin (`/api/admin`) — All routes require Admin role

Method	Endpoint	Description
GET	`/dashboard-stats`	KPI data
GET	`/dashboard-analytics`	Chart data
GET	`/search`	Global search
GET/POST/PUT/DELETE	`/roles`	Role management
GET	`/permissions`	All permissions
GET/PUT	`/roles/:id/permissions`	Role permissions
GET	`/modules`	System modules
GET	`/staff`	Staff list

Method	Endpoint	Description
GET/POST/PUT/DELETE	`/users`	User management
GET	`/audit-logs`	Audit trail

Other Routes

Base	Method	Endpoint	Description
`/api/medical-records`	POST	`/`	Create clinical record
`/api/medical-records`	GET	`/:patientId`	Patient history
`/api/prescriptions`	GET/POST	`/`	List/save prescriptions
`/api/prescriptions`	GET	`/patient/:id`	Patient prescriptions
`/api/prescriptions`	PATCH	`/:id/status`	Update status
`/api/tests`	GET/POST/DELETE	`/`	Test master CRUD
`/api/inventory`	—	Various	Products, indents, issues
`/api/timeline`	GET	`/`	Patient timeline
`/api/daily-op`	GET	`/`	Daily OP report
`/api/search`	GET	`/`	Context-aware search
`/api/reports`	GET	`/generate`	PDF report generation
`/api/doctors`	GET/PUT	`/`, `/:id`	Doctor list & status

9. Database

- **Engine:** MySQL 8.0
- **Schema File:** `server/hms.sql` (1.2 MB full dump)
- **Connection:** Pool with 100 max connections, keep-alive, IST timezone (`+05:30`), `utf8mb4` charset
- **Named Placeholders:** Enabled for cleaner queries
- **Multiple Statements:** Disabled (SQL injection prevention)

Key Tables (Inferred from modules)

Category	Tables
Auth	`users`, `roles`, `permissions`, `role_permissions`
Patients	`patients`
Clinical	`medical_records`, `prescriptions`, `prescription_medicines`
Appointments	`appointments`, `appointment_transactions`
IPD	`beds`, `admissions`, `encounters`
OPD	`opd_visits`
Lab	`test_master`, `test_entry`, `lab_billing`, `lab_requests`, `lab_doctors`
Pharmacy	`pharma_products`, `pharma_vendors`, `pharma_purchases`, `pharma_enquiries`, `pharma_requests`, `pharma_alerts`, `suppliers`, `stock_orders`, `stock_order_items`
Billing	`hospital_bills`, `bill_details`, `pharma_bills`
Inventory	`products`, `indent_entry`, `issue_entry`
Payroll	`employees`, `attendance`, `shifts`, `salaries`, `payslips`
System	`audit_logs`, `sidebar_modules`

---

10. Middleware Pipeline

Request flows through this ordered pipeline:

```
1. Helmet          → Security headers (XSS, HSTS, etc.)
2. Compression     → gzip responses (threshold: 1KB, level: 6)
3. CORS            → Dynamic origin validation (env-driven whitelist)
4. Morgan→Winston  → HTTP request logging (skips /health)
5. Body Parser     → JSON + URL-encoded (10MB limit)
6. Auto Sanitize   → Strip HTML/XSS from all inputs
7. Rate Limiters:
   → Global:  500 req/min/IP (all /api routes)
   → Auth:    20 req/15min/IP (login only)
   → Write:   100 req/min/IP (mutations)
   → Reports: 10 req/min/IP (heavy operations)
8. Route Handlers  → Module-specific logic
9. Error Handler   → Global catch-all error middleware
```

Caching Middleware

- **In-memory response cache** with configurable TTL per route
- `cacheResponse(seconds)` — Caches GET responses
- `invalidateCache(pattern)` — Clears cache on write operations

---

11. Reusable Components

Common UI Components (`client/src/components/common/`)

Component	Description
`Button.jsx`	Styled button with variants
`Card.jsx`	Content card wrapper
`Badge.jsx`	Status/label badges
`Input.jsx`	Form input component
`DataTable.jsx`	Generic table with sorting
`DashboardStack.jsx`	KPI widget stack for dashboards
`MasterSearch.jsx`	Global search with context-aware results
`MedicineSearch.jsx`	Autocomplete for medicine names

Component	Description
`PatientSearch.jsx`	Patient lookup with inline results
`PrintLayout.jsx`	Print-optimized wrapper

Print Templates (`client/src/components/print/`)

Component	Description
`InvoicePrint.jsx`	Hospital invoice print layout
`PatientHistoryPrint.jsx`	Patient history report print
`PharmacyBillPrint.jsx`	Pharmacy bill print layout

---

12. DevOps & Deployment

Docker

`docker-compose.yml` runs 3 services:

Service	Container	Port	Details
`backend`	hms-backend	3000	Node.js Express server
`frontend`	hms-frontend	80	Nginx serving React build
`db`	hms-db	3307→3306	MySQL 8.0 with persistent volume

Kubernetes (`k8s/`)

Full production-grade K8s manifests:

File	Resource
`01-namespace.yaml`	`hms` namespace
`02-secrets.yaml`	DB credentials, JWT secret

File	Resource
`03-configmaps.yaml`	App configuration
`10-db.yaml`	MySQL Deployment + PVC + Service
`20-backend.yaml`	Backend Deployment + Service (health checks, resource limits)
`30-frontend.yaml`	Frontend Deployment + Service
`40-ingress.yaml`	Ingress routing

Nginx Configuration

The client includes `nginx.conf` for production serving with:

- SPA fallback routing
- Static asset caching
- Reverse proxy to backend API

---

13. Testing

Type	Location	Command
Load Testing	`server/tests/load/`	`npm run load-test` (autocannon)
Sanity Check	`server/tests/sanity/`	`npm run sanity-check`
E2E Verification	`server/tests/e2e/`	`npm run e2e-verify`

---

14. Environment Configuration

Server `.env` Variables

```
# Server
PORT=5000
```

```
NODE_ENV=development
ENABLE_CLUSTER=false
CLUSTER_WORKERS=4

# Database
DB_HOST=localhost
DB_USER=root
DB_PASS=root
DB_NAME=hms
DB_PORT=3306
DB_POOL_SIZE=100

# Auth
JWT_SECRET=your_secret_key

# CORS
CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

NPM Scripts

Server:

Script	Command	Description
`dev`	`nodemon server.js`	Development with hot reload
`start`	`nodemon server.js`	Start server
`prod`	`NODE_ENV=production node server.js`	Production mode
`fix-schema`	`node src/scripts/fix_schema.js`	Fix DB schema issues
`create-indexes`	`node src/scripts/create_indexes.js`	Create DB indexes

Client:

Script	Command	Description
`dev`	`vite`	Dev server (port 5173)
`build`	`vite build`	Production build
`preview`	`vite preview`	Preview production build

15. Role Workflows

Detailed workflow specs exist for each role:

Role	File
Admin	`Admin Workflow.txt`
Doctor	`Doctor Workflow.txt`
Receptionist	`Receptionist Workflow.txt`
Lab Technician	`Lab Technician Workflow.txt`
Pharmacist	`Pharma Workflow.txt`

Admin Day-in-Life Summary

1. **08:00 AM** — Login. Check dashboard for critical stock alerts.
2. **09:00 AM** — User management. Activate/deactivate accounts.
3. **11:00 AM** — Financial oversight. Approve refunds/discounts.
4. **02:00 PM** — Master updates. Lab test prices, new medicine entries.
5. **05:00 PM** — Audit. Review voided bills for fraud prevention.
6. **07:00 PM** — Reports. Generate daily collection & census reports.
7. **07:30 PM** — Day close. Freeze financials. Logout.

Appendix: Existing Documentation Files

File	Description
`README.md`	Project overview & setup guide
`documentation/API_GUIDE.md`	Core API documentation with examples
`documentation/PROJECT_HISTORY.md`	Development timeline
`documentation/hms_postman_collection.json`	Postman collection for API testing

File	Description
`PAYROLL_MODULE_BLUEPRINT.md`	Detailed payroll/HRMS architecture
`DEPLOYMENT_STATUS.md`	Deployment status tracking
`server/SETUP_GUIDE.md`	Server setup instructions
`client/SETUP_GUIDE.md`	Client setup instructions
`k8s/README.md`	Kubernetes deployment guide
`k8s/OPERATIONS.md`	K8s operational runbook